

Answers — Async JavaScript: Promises

Teacher-facing document. Model answers and grading notes for each exercise.

Exercise 1 — Your First Article

Model answer:

```
const url = "https://articles-zxjs.onrender.com/articles/1";

fetch(url)
  .then(response => response.json())
  .then(data => console.log(data.data.title + " by " +
    data.data.author))
  .catch(error => console.log("Error:", error));
```

Key elements a correct answer includes:

- `response.json()` returned (not just called) in the first `.then`
- `data.data.title` and `data.data.author` correctly accessed (note the nested `data.data` — a common stumble)
- A `.catch()` present — even a minimal one

Grading note:

- **Weak:** Student accesses `data.title` directly (missing the nested `data.data` wrapper). They log undefined. They may not notice without checking the console carefully.
 - **Adequate:** All three points above present and working.
 - **Strong:** Student notices that `.json()` itself returns a Promise, and can explain why `return response.json()` is necessary for the next `.then` to receive the parsed object.
-

Exercise 2 — Chain It

Model answer:

```
const baseUrl = "https://articles-zxjs.onrender.com";

fetch(baseUrl + "/articles")
  .then(response => response.json())
```

```

.then(list => fetch(baseUrl + list.data[0]))
.then(response => response.json())
.then(article => console.log("First article: " +
    article.data.title + " by " + article.data.author))
.catch(error => console.log("Could not load article:",
    error.message));

```

Key elements a correct answer includes:

- Four chained `.then()` calls (parse list → fetch article → parse article → log)
- `list.data[0]` correctly used to extract the first article URL
- Base URL prepended before the second fetch
- Formatted output matching the specification
- `.catch()` with a meaningful message

Grading note:

- **Weak:** Student nests the second fetch inside the first `.then` callback — this works but reproduces the callback-hell pattern the lesson is meant to solve. Address directly: “Does your chain stay flat?”
 - **Adequate:** Four sequential `.then()` calls, correct data path, working output.
 - **Strong:** Student returns the result of the second fetch explicitly and can explain that returning a Promise from `.then` is what makes flat chaining possible — the next `.then` waits for that inner Promise to settle.
-

Exercise 3 — Pick Your Article

Model answer (example with article 7):

```

const url = "https://articles-zxjs.onrender.com/articles/7";

fetch(url)
  .then(response => response.json())
  .then(data => {
    const { id, title, intro } = data.data;
    console.log(`Article ${id}: ${title} - ${intro.slice(0,
        60)}...`);
  })
  .catch(error => console.log("Error fetching article:",
    error.message))
  .finally(() => console.log("Request complete.));

```

Key elements a correct answer includes:

- A different article ID from Exercises 1 and 2
- `.slice(0, 60)` (or equivalent) to truncate the intro
- `.finally()` present and unconditional

Grading note:

- **Weak:** Student hardcodes the intro string rather than truncating it dynamically, or copies the ID from a previous exercise.

- **Adequate:** All points above; `.finally()` fires correctly in both success and error scenarios.
 - **Strong:** Student tests the `.finally()` by temporarily breaking the URL and confirms it still fires. They can articulate why `.finally()` is useful (cleanup, independent of outcome — e.g., hiding a loading spinner).
-

Exercise 4 — Multiple Articles at Once

Model answer:

```
const baseUrl = "https://articles-zxjs.onrender.com";

const article1 = fetch(baseUrl + "/articles/1")
  .then(r => r.json())
  .then(d => d.data.title);

const article5 = fetch(baseUrl + "/articles/5")
  .then(r => r.json())
  .then(d => d.data.title);

const article10 = fetch(baseUrl + "/articles/10")
  .then(r => r.json())
  .then(d => d.data.title);

Promise.all([article1, article5, article10])
  .then(([title1, title5, title10]) => {
    console.log(`Articles: ${title1} | ${title5} | ${title10}`);
  })
  .catch(error => console.log("One or more requests failed:",
    error.message));
```

Key elements a correct answer includes:

- Three separate fetch chains, each resolving to the title string (not the raw response)
- `Promise.all()` wrapping all three
- Destructuring in the `.then([a, b, c])` handler
- A `.catch()` — and ideally an understanding that `Promise.all` rejects as soon as *any* Promise rejects

Grading note:

- **Weak:** Student chains all three fetches sequentially (each inside the previous `.then`) — this works but is not parallel. The requests run one after another, not simultaneously.
 - **Adequate:** Parallel requests, `Promise.all`, correct destructuring, `.catch` present.
 - **Strong:** Student can explain that `Promise.all` has “fail-fast” behaviour — if one request fails, the `.catch` fires immediately even if the other requests eventually succeed. They may also explore `Promise.allSettled` as an alternative that waits for all outcomes regardless of failure.
-

Exercise 5 — Handle the Storm

Expected observations per scenario:

1. **Non-existent article (/articles/999):** The API returns a 404 response. `fetch` itself does *not* reject — it resolves with a non-OK response. `.catch()` does NOT fire unless the student explicitly checks `response.ok`. This is a valuable discovery: an HTTP error $\neq$ a rejected Promise.
2. **Wrong domain:** `fetch` rejects with a `TypeError: Failed to fetch`. `.catch()` fires. This is a true network-level failure.
3. **Thrown error in `.then()`:** `.catch()` catches it, regardless of where in the chain it is placed. The error propagates down the chain until the first `.catch()`.
4. **`.finally()` behaviour:** Fires in all three scenarios — both on success and on failure.

Key elements a correct answer includes:

- Comments in code for each scenario
- Correct identification of the `response.ok` gap (scenario 1)
- Confirmation that `.finally()` always runs

Grading note:

- **Weak:** Student only tests scenario 3 (throw in `.then`). Misses the important distinction in scenario 1 — the most practically relevant finding.
 - **Adequate:** All four scenarios tested with comments.
 - **Strong:** Student adds a `response.ok` check (`if (!response.ok) throw new Error("HTTP error: " + response.status)`) after discovering scenario 1's behaviour, and explains why this matters in real applications.
-

Exercise 6 — Promises vs. Callbacks

Model answer (callback version using `XMLHttpRequest`, equivalent to Exercise 2):

```
const xhr1 = new XMLHttpRequest();
xhr1.open("GET", "https://articles-zxjs.onrender.com/articles");
xhr1.onload = function() {
  if (xhr1.status === 200) {
    const list = JSON.parse(xhr1.responseText);
    const articlePath = list.data[0];

    const xhr2 = new XMLHttpRequest();
    xhr2.open("GET", "https://articles-zxjs.onrender.com" +
      articlePath);
    xhr2.onload = function() {
      if (xhr2.status === 200) {
        const article = JSON.parse(xhr2.responseText);
```

```

        console.log("First article: " + article.data.title + " by " + article.data.author);
    } else {
        console.log("Error fetching article: " + xhr2.status);
    }
};
xhr2.onerror = function() { console.log("Network error on second request"); };
xhr2.send();
} else {
    console.log("Error fetching list: " + xhr1.status);
}
};
xhr1.onerror = function() { console.log("Network error on first request"); };
xhr1.send();

```

Key elements a strong written answer includes:

- Identifies readability / flat structure as the main advantage of Promises
- Mentions error handling centralisation (one `.catch()` vs. error callbacks at every nesting level)
- Acknowledges a case where callbacks are still valid (simple one-off operations, event listeners, legacy codebases)
- Does not just say “Promises are always better” — there are tradeoffs

Grading note:

- **Weak:** Student writes “Promises are better because they look nicer.” No argument, no tradeoff considered.
- **Adequate:** Identifies at least two concrete advantages of Promises, acknowledges one situation where callbacks remain appropriate.
- **Strong:** Student discusses composability (chaining, `Promise.all`), error propagation, and perhaps notes that callbacks are still the right model for event listeners (`addEventListener`) where Promises don’t fit — showing genuine transfer of understanding.